

A

SOFTWARE PROJECT

ON

“Project name”

MULTI-GAME WEB APP PORTAL

SUBMITTED TO:



LJ UNIVERSITY AHMEDABAD

Master of Science (Information Technology) Integrated

GUIDED BY:

Prof. Nilesh Parmar

PROJECT BY:

Name: Dhvanit Shah, Kavy Sharma, Harmeet sinha

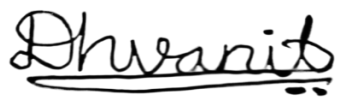
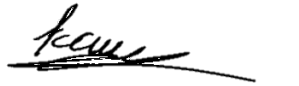
Enrollment No: 24004500210254,24004500210261,24004500210266

PLAGIARISM DECLARATION

To whom so ever it may concern

I/ We, the project team, confirm that this assignment is my own work, is not copied from any other person's work (published or unpublished), and has not previously sub- mitted for assessment either at University or elsewhere. I/We confirm that I/we have read and understood the rules regulations on plagiarism in LJ University.

Date:

Enrollment no	Name	Sign
24004500210254	Dhvanit Shah	
24004500210261	Kavy Sharma	
24004500210266	Harmeet sinha	

MULTI-GAME WEB APP PORTAL

Sr. No.	TABLE OF CONTENTS	Page No.
1.	INTRODUCTION	4
	1.1 Existing System	5
	1.2 Need for the New System	5
	1.3 Objective of the New System	6
	1.4 Problem Definition	6
	1.5 Core Components	7
	1.6 Project Profile	8
	1.7 Assumptions and Constraints	8
	1.8 Advantages and Limitations of the Proposed System	9
2.	REQUIREMENT DETERMINATION & ANALYSIS	10
	2.1 Requirement Determination	10
	2.2 Targeted Users	12
	2.3 Hardware and Software Requirements	12
3.	SYSTEM DESIGN	14
	3.1 Use Case Diagram	14
	3.2 Data Flow Diagram (DFD)	15
	3.3 Interaction Diagram	16
	3.4 Activity Diagram	17
	3.5 Data Dictionary	18
4.	DEVELOPMENT	23
	4.1 Coding Standard	23
	4.2 Screenshots	26
5.	PROPOSED ENHANCEMENTS	27
6.	CONCLUSION	28
7.	BIBLIOGRAPHY	28

1. INTRODUCTION

The rapid evolution of the digital entertainment industry has transformed the way people discover and interact with online games, and the web gaming sector is no exception. Traditional web gaming often involves fragmented experiences, limited progress tracking, and time-consuming searches across multiple disconnected websites. To overcome these challenges and provide a seamless interactive experience, our project introduces the Multi-Game Web App Portal, a unified platform designed to serve administrators, game developers, and players under one comprehensive system.

The Multi-Game Web App Portal is built with the objective of simplifying the gaming experience while enhancing engagement and accessibility for all stakeholders. For players, the portal offers a diverse library of games spanning across various categories such as action, puzzles, strategy, and more. With intuitive navigation, secure user authentication, leaderboard rankings, and achievement tracking, players can effortlessly immerse themselves in their favorite games directly from their web browsers. The system also integrates features like game reviews, score submissions, and profile customization to provide a deeply interactive and rewarding gaming journey.

Developers benefit from dedicated dashboards that enable them to upload their game creations, manage publication statuses, and monitor player engagement. This empowers independent creators to easily expand their audience and showcase their talent while maintaining full control over their game portfolios. On the other hand, administrators are equipped with powerful management tools for monitoring user activities, reviewing developer game submissions, and ensuring a safe, high-quality environment. The inclusion of centralized approval workflows and category management ensures effective system governance and a curated gaming catalog.

In essence, the Multi-Game Web App Portal is more than just a collection of web games; it is a dynamic digital ecosystem that connects creators and players in a user-friendly, secure, and competitive environment. By streamlining publication for developers, providing administrators with robust oversight tools, and offering players a centralized hub for endless entertainment, the portal aims to redefine the web gaming landscape in the modern digital era.

1.1 Existing System

In the existing system, web gaming is mainly done through scattered, standalone websites or by downloading multiple independent applications with limited community features. Players face challenges like fragmented progress tracking, a lack of centralized leaderboards, and the inconvenience of managing multiple accounts across different platforms. Game developers often struggle with complex deployment processes, finding a reliable platform to easily publish lightweight web games, and reaching a wider player base, while administrators have minimal control over content curation, user analytics, and platform wide management.

1.2 Need for the New System

The growing demand for accessibility, community engagement, and centralized tracking in online gaming highlights the need for a modern system. Players want easy access to a diverse catalog of games, secure user profiles, and real-time leaderboard rankings alongside achievements. Developers require a streamlined publishing process and better insights into player engagement, while administrators need powerful tools to curate content and oversee platform operations. A unified digital platform like the Multi-Game Web App Portal addresses

these challenges, creating a seamless and interactive experience for all users.

1.3 Objective for the New System

The main objective of the Multi-Game Web App Portal is to provide a unified digital platform that simplifies game discovery, gameplay, and content management. It aims to deliver a convenient, secure, and engaging experience for players, an efficient game publication and performance tracking workflow for developers, and robust moderation tools for administrators. By integrating advanced features like interactive dashboards, real-time leaderboards, achievement systems, and centralized reviews, the system seeks to enhance community engagement, platform security, and overall user satisfaction.

1.4 Problem Definition

Traditional web gaming ecosystems often lack centralization, community connectivity, and transparency. Players face issues like scattered game libraries, unrecorded achievements, and the hassle of switching between independent sites, while developers struggle with complicated publishing infrastructure and reaching a dedicated audience. Administrators also lack centralized tools for content moderation and cross-game analytics. These limitations highlight the need for a unified digital platform that streamlines game distribution, playing, and community management.

1.5 Core Components of the Project

1. **Admin Module** – Manages user accounts, oversees game categories, and handles the approval process for new games submitted by developers.
2. **Developer Module** – Allows game creators to register, upload their games, manage game details, and track player reviews and engagement.
3. **Player Module** – Provides players with secure registration, game browsing, the ability to submit ratings and reviews, and a personal profile.
4. **Game & Leaderboard System** – Handles gameplay integration and dynamically updates global and game-specific high scores.
5. **Achievement System** – A dedicated module that defines game-specific milestones and tracks player progress as they unlock achievements.
6. **Dashboard & Analytics** – Offers visual reports on user activity, most played games, and developer portfolio performance for better curation.
7. **Authentication & Security** – Ensures secure encrypted logins with strict role-based access control (Admin, Developer, Player) across the MERN stack.

1.6 Project Profile

Group No	IMSCIT SEM-5
Project Title	Multi-Game-WebApp
Project Technology	Frontend: React js, HTML5, CSS3, JavaScript Backend: Node.js, Express.js
Database	MongoDB
Team Members	Dhvanit Shah Kavy Sharma Harmeet sinha
Internal Guide	Prof. Nilesh Parmar

1.7 Assumptions and Constraints

Assumptions

1. Users (admins, developers, and players) have access to a stable internet connection and modern web browsers.
2. Developers will provide fully functional, safe, and optimized HTML5 web-based games along with accurate descriptions.
3. Players are familiar with standard web interfaces and basic online game controls.

Constraints

1. The system depends heavily on internet connectivity; offline gameplay is not supported.
2. Heavy graphical games may experience performance bottlenecks depending on the player's web browser and device hardware capabilities.

3. Strict data security and anti-cheat measures must be enforced across leaderboards, which may limit certain integrations or features for developers.

1.8 Advantages and Limitations of the Proposed System

Advantages:

- Provides a single, unified platform connecting admins, game developers, and players.
- Simplifies game publication, leaderboard tracking, and community management.
- Offers real-time analytics on player engagement and most popular games to assist developers.
- Improves player convenience by centralizing game discovery, achievements, and reviews in one place.

Limitations:

- Requires a stable and relatively fast internet connection for loading game assets and smooth gameplay.
 - Complex or graphic-intensive web games may face performance limitations depending on the user's browser and hardware.
 - May face scaling challenges and server latency if a massive number of concurrent players attempt to sync scores simultaneously.
-

2. Requirement Determination and Analysis

The proposed system focuses on meeting the needs of admins, developers, and players. Admins require tools for managing users, approving developer submissions, overseeing game categories, and generating reports, while developers need features for game publication and monitoring player engagement. Players require easy registration, game browsing, achievement tracking, submitting reviews, and participating in global leaderboards. The system must ensure high performance, security, and scalability, while offering user-friendly dashboards and efficient workflows for smooth operations across all roles.

2.1 Functional Requirements:

1. **User Authentication:** The system must provide secure login, registration, and profile management for Admins, Developers, and Players.
2. **Game Management:** Developers must be able to submit games along with details like titles, descriptions, and media, while Admins must be able to approve, reject, or remove these games.
3. **Gameplay & Progression:** Players should be able to easily launch and play games, unlock predefined achievements, and submit their high scores to global leaderboards.
4. **Review System:** The system must allow registered players to rate games out of 5 stars and leave written feedback or comments.
5. **Dashboard & Analytics:** Admins and Developers should have access to interactive dashboards to view active user metrics, game popularity, and player engagement reports.

6. **Search & Filters:** The portal should allow players to easily search for specific games and filter the library by genres, categories, or specific developers.

Non-Functional Requirements:

1. **Performance:** The portal should handle multiple concurrent players and provide fast response times for game loading, API requests, and real-time leaderboard updates.
2. **Scalability:** The system must easily support growth in terms of the game library, registered users, and concurrent gameplay sessions without performance degradation.
3. **Security:** All user data, passwords, and API endpoints must be protected using robust encryption (e.g., JWT) to secure accounts and prevent leaderboard manipulation or cheating.
4. **Usability:** The interface should be highly interactive, user-friendly, and responsive, ensuring a seamless experience across modern desktop and mobile web browsers.
5. **Reliability:** The system must ensure minimal downtime and guarantee accurate data consistency, particularly for player scores and unlocked achievements.
6. **Maintainability:** The application's MERN stack codebase should be heavily modular and component-based, making it easy to introduce new features, add games, or apply bug fixes.

2.2 Targeted Users

The system targets the following user groups:

- **Admins** – to manage user accounts, review and approve new game submissions, oversee categories, and monitor overall platform health.
- **Developers** – to register, upload and manage their games, track player reviews, and monitor engagement with their content.
- **Players** – to browse the game library, play games, unlock personal achievements, submit high scores to global leaderboards, and write reviews.

2.3 Hardware and Software Requirements

Hardware Requirements (Minimum)

Client-Side (Players & Admins):

- Processor: Intel Core i3 or equivalent (1.5 GHz or higher)
- RAM: 4 GB or higher
- Storage: Minimal local storage (as the application is web-based)
- Display: 1366x768 resolution or higher
- Internet Connection: Stable broadband connection (minimum 2 Mbps for smooth gameplay)

Server-Side (Development/Hosting):

- Processor: Dual-core CPU (2.0 GHz or higher)
- RAM: 8 GB or higher recommended (for running Node.js and MongoDB concurrently)
- Storage: 20 GB SSD or higher for database expansion and game asset storage

Software Requirements

Client-Side (End User):

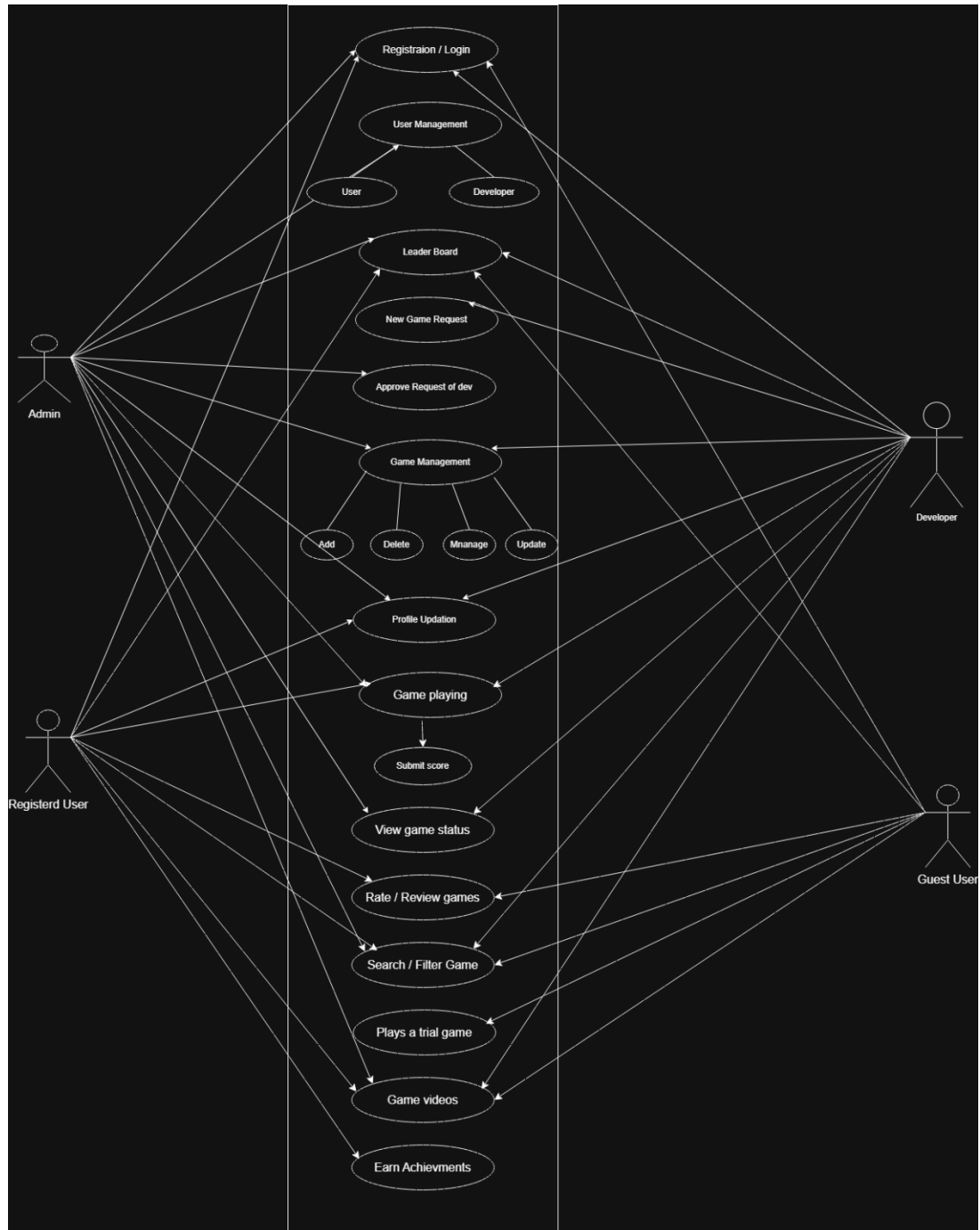
- Operating System: Windows, macOS, Linux, Android, or iOS
- Web Browser: Modern browsers supporting HTML5 and JavaScript (e.g., Google Chrome v80+, Mozilla Firefox v75+, Safari v13+, or Microsoft Edge)

Server-Side (Development Environment):

- Operating System: Windows, macOS, or Linux
 - Environment: Node.js (v14.x or higher)
 - Database: MongoDB (v4.x or higher) or MongoDB Atlas (Cloud)
 - Frontend Framework: React.js (v18+)
 - Web Server/Framework: Express.js
 - Tools/IDE: Visual Studio Code, Git, Postman (for API testing)
-

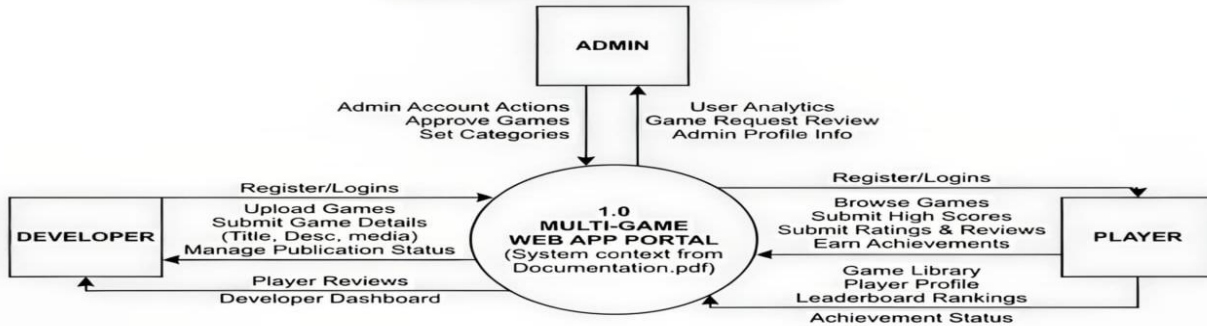
3. SYSTEM DESIGN

3.1 Use Case Diagram

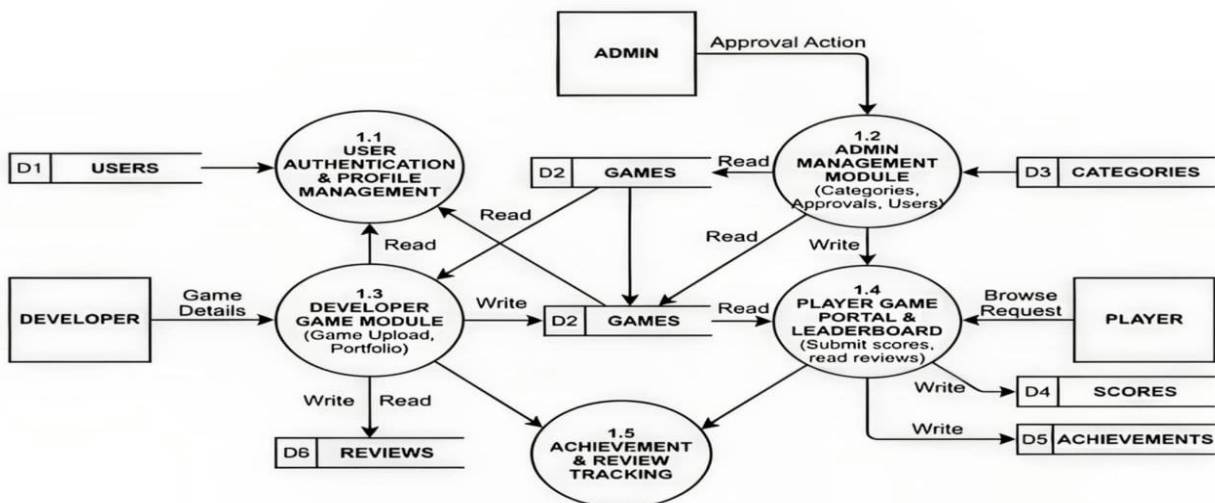


3.2 Data Flow Diagram (DFD)

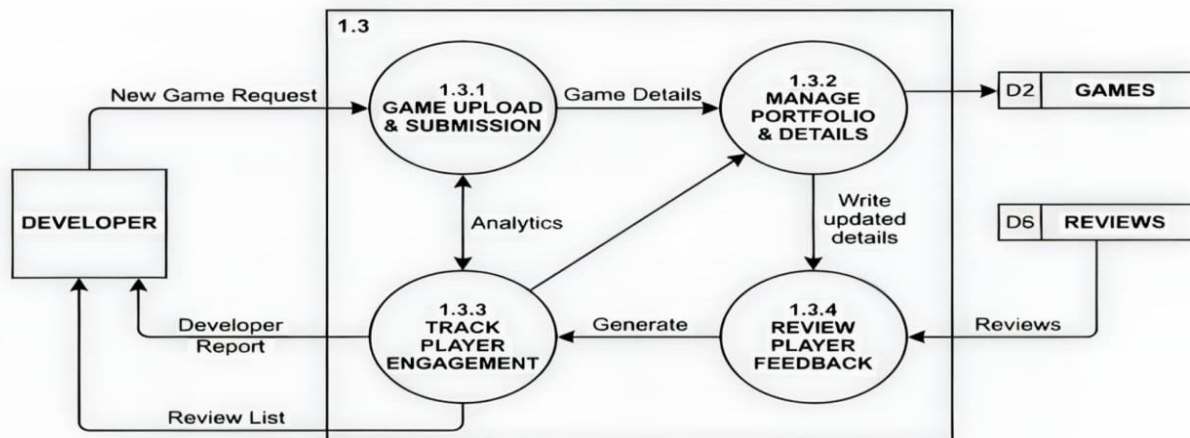
DFD LEVEL 0: CONTEXT DIAGRAM



DFD LEVEL 1: DETAILED PROCESS DIAGRAM - MULTI-GAME WEB APP PORTAL
(Decomposition of System 1.0 from Image 0)



DFD LEVEL 2: DECOMPOSITION DIAGRAM - PROCESS 1.3 DEVELOPER GAME MODULE
(Expanded view of Process 1.3 from Image 1)



3.3 Interaction Diagram

3.4 Activity Diagram

3.5 Data Dictionary

1. User Management (users)

This table stores core account information for all actors identified in the diagram (Admin, Registered User, Developer).

Field Name	Data Type	Note	Description
id	BIGINT	PK, Auto-increment	Unique user identifier.
role	ENUM	admin, player, developer	User permission level (Extended to include Developer).
name	VARCHAR(255)		User's display name.
email	VARCHAR(255)	Unique	Login email address.
password	VARCHAR(255)	Hashed	Encrypted login password.
avatar_url	VARCHAR(255)	Nullable	Path to profile picture for "Profile Updation" use case.

2. Game Inventory (games)

The repository for games, including their publication status which is managed by Admins and requested by Developers.

Field Name	Data Type	Note	Description
id	BIGINT	PK, Auto-increment	Unique game identifier.
title	VARCHAR(255)		Name of the game.
category_id	BIGINT	FK (categories.id)	Link to categories table.
developer_id	BIGINT	FK (users.id)	Links the game to the Developer who submitted it.
description	TEXT		Detailed game info.
image_url	VARCHAR(255)		Path to thumbnail image.
video_url	VARCHAR(255)		Path to gameplay video for "Game videos" use case.
game_file	VARCHAR(255)		Path to game logic (entry).
status	ENUM	public, private, pending	"Pending" added for the "Approve Request of dev" use case.

3. Categories (categories)

Organizes games into genres to facilitate the "Search / Filter Game" use case.

Field Name	Data Type	Note	Description
id	BIGINT	PK, Auto-increment	Unique category identifier.
name	VARCHAR(100)	Unique	Genre name (e.g., Action, Puzzle).
slug	VARCHAR(100)	Unique	URL-friendly label.

4. Game Reviews (reviews)

Handles the "Rate / Review games" use case for registered users.

Field Name	Data Type	Note	Description
id	BIGINT	PK, Auto-increment	Unique review ID.
user_id	BIGINT	FK (users.id)	The author of the review.
game_id	BIGINT	FK (games.id)	The game being reviewed.
rating	TINYINT	Range 1-5	Numeric score.
comment	TEXT		Written feedback.

5. Leaderboard & Scoring (scores)

Tracks performance for the "Submit score" and "Leader Board" use cases.

Field Name	Data Type	Note	Description
id	BIGINT	PK, Auto-increment	Unique record ID.
user_id	BIGINT	FK (users.id)	The player.
game_id	BIGINT	FK (games.id)	The game played.
score	BIGINT		Achievement value.

6. Achievements (achievements) — NEW

Supports the "Earn Achievements" use case shown in the diagram.

Field Name	Data Type	Note	Description
id	BIGINT	PK, Auto-increment	Unique achievement identifier.
game_id	BIGINT	FK (games.id)	The game this achievement belongs to.
title	VARCHAR(255)		Name of the achievement (e.g., "First Win").
criteria	INT		Score or action required to unlock.

7. User Achievements (user_achievements) — NEW

A many-to-many table to track which users have earned which achievements.

Field Name	Data Type	Note	Description
user_id	BIGINT	FK (users.id)	The player who earned it.
achievement_id	BIGINT	FK (achievements.id)	The specific achievement earned.
earned_at	TIMESTAMP		Date and time of the achievement.

Updated Entity Relationship Summary

- **Users 1:M Reviews:** A user can write many reviews.
 - **Users 1:M Scores:** A user can have many scores.
 - **Users M:N Achievements:** Users earn many achievements; achievements are earned by many users.
 - **Developers 1:M Games:** A developer can submit multiple games for approval.
 - **Categories 1:M Games:** A category has many games.
 - **Games 1:M Reviews/Scores:** Games host multiple reviews and leaderboard entries.
-

4. DEVELOPMENT

4.1 Coding Standard

1. Naming Conventions

Use meaningful and descriptive names for variables, functions, components, and other identifiers.

Use camelCase for variables and functions (e.g., fetch Games, player Score).

Use Pascal Case for class names, React components, and database models (e.g., Game Card, User Schema, AUTH Controller).

Avoid abbreviations unless they are widely understood within the development context.

2. Commenting and Documentation

Write clear, concise comments explaining the purpose and logic of complex React components and backend API endpoints.

Use JSDoc-style comments (e.g., `/** ... */`) to describe functions, API routes, parameters, and return values.

Update comments and documentation whenever code changes to keep them accurate.

Include a brief file header with the author, purpose, and description for major module and configuration files.

3. Code Structure and Formatting

Indent code blocks consistently (e.g., 2 spaces per indent level is standard for JavaScript and React).

Use line breaks to separate logical sections of code, React hooks, and API endpoints.

Keep line length within 80-120 characters for readability; utilizing automated formatters like Prettier is highly recommended.

Group related state variables, lifecycle methods, and helper functions logically.

Use consistent brace styles (e.g., opening brace on the same line).

4. Error Handling and Validation

Validate all external inputs and user data on both the frontend (React) and the backend API (Node.js/Express).

Use try/catch blocks with async/await to handle promise rejections and failures gracefully.

Ensure the API always returns proper HTTP status codes (e.g., 400 for bad requests, 404 for not found, 500 for server errors).

Log backend errors with sufficient context to enable debugging, while ensuring sensitive stack traces are not exposed to the client.

5. Code Efficiency and Optimization

Avoid redundant code by strictly reusing React components and backend helper functions.

Write highly modular and reusable code, leveraging modular imports and exports in Node.js.

Optimize algorithms for time and space complexity where necessary, and use React performance hooks (e.g., use Memo, use Callback) for heavy computations while prioritizing readability.

6. Security Best Practices

Sanitize and validate all external inputs to prevent Cross-Site Scripting (XSS) on the frontend and NoSQL injections in MongoDB.

Never hardcode sensitive data such as JWT secrets, database URIs, or API keys in the source code; strictly use environment (.env) variables.

Implement proper CORS policies, rate limiting, and follow MERN stack-specific security guidelines.

7. Version Control and Comments

Use clear, meaningful Git commit messages describing what changes were made and why.

Keep code merges clean by resolving conflicts carefully and using dedicated branches for new features or bug fixes.

Tag major releases and critical updates for future reference and deployment tracking.

4.2 Screenshots



5. PROPOSED ENHANCEMENTS

Enhanced Security: Implement rate limiting on authentication API endpoints and transition to using HTTP-only, secure cookies for JWT token storage to mitigate XSS vulnerabilities and session hijacking.

Database Improvements: Expand the Game and User MongoDB schemas to track advanced metrics, such as total hours played, last login timestamps, and detailed, granular achievement progress.

Gameplay & Leaderboards: Upgrade the leaderboard system by integrating Web Sockets (Socket.io) to support real-time updates during matches, ensuring high scores are instantly synced across all active clients.

Admin Dashboard Upgrades: Add robust filters, CSV export options, and dynamic visual charts tracking platform growth, most popular genres, and developer portfolio metrics to drive better content curation.

Developer Features: Allow developers to access detailed player retention analytics, deploy new game version updates without downtime, and temporarily hide their games from the public catalog for maintenance.

Player Experience: Introduce social features such as adding friends, direct messaging, user-to-user challenges, and personalized game recommendations based on past gameplay history.

Code Structure: Incrementally migrate the codebase to strict TypeScript across both the frontend and backend for better type safety, and implement Redux for complex global state management in React.

Centralized Configuration: Consolidate application constants (API routes, game asset upload limits, and pagination defaults) into a dedicated, environment-aware configuration utility for highly scalable maintenance.

6. CONCLUSION

The Multi-Game Web App project successfully delivers a multi-role interactive portal with secure authentication, dedicated dashboards, and a highly responsive UI tailored for administrators, game developers, and players.

Core features such as user registration, secure logins, developer game publication workflows, interactive gameplay, global leaderboards, and achievement tracking are fully implemented using the modern MERN stack (MongoDB, Express.js, React, and Node.js).

The system actively demonstrates the practical application of RESTful API design, robust token-based authentication (JWT), complex NoSQL data modeling, and modern component-driven frontend architecture.

The current implementation establishes a strong technical foundation for future enhancements, paving the way for real-time multiplayer integrations, advanced player analytics, deeper social features, and expansive game catalogs.

Overall, the project comprehensively meets the objectives of a centralized web gaming platform and offers ample scope for future continuous improvement, scalability, and community growth.

7. BIBLIOGRAPHY

React Documentation – Conceptual references for functional components, custom hooks, and virtual DOM rendering used to build the interactive frontend interface.

Node.js & Express.js Documentation – Technical guides for setting up the backend server environment, routing RESTful API endpoints, and handling authentication middleware.

MongoDB & Mongoose Documentation – References for NoSQL database schema design, data modeling, and executing queries for users, game catalogs, and leaderboards.

MDN Web Docs – Core HTML5, CSS3, and modern JavaScript (ES6+) references used for building responsive layouts, handling asynchronous API calls, and form validation.

Font Awesome / UI Library Documentation – Styling and icon usage guidelines for integrating visual elements into navigation headers, action buttons, and dashboard analytics cards.

Stack Overflow and Developer Forums – Conceptual guidance for resolving technical challenges related to JWT authentication flows, React state management, and MongoDB aggregations (no direct code copied; strictly used for debugging insights and architectural approaches).